

INFORME TECNICO ANALITICO

Integrantes:

- Ezequiel Benito Tomas Veloso 45.650.965
- Facundo Agustin Moreira 43346675
- Juliana Maillen Riquelme Griffith 46395489
- Fabregat Malena Ayelen 40279603
- Zacarias Matias Juan Ezequiel – 43930609

Tema Investigado:

Modelado e implementación de un sistema de control para semáforos utilizando paradigmas de programación funcional. Implementando funciones puras, inmutabilidad y composición funcional para resolver un problema de ingeniería de sistemas. Integración de librerías externas, análisis comparativo de inferencia de tipos y estructuras orientadas a expresiones utilizando Common Lisp y OCaml.

Índice:

1. Introducción y fundamentos del diseño funcional adoptado.
2. Justificación de herramientas y arquitectura de frontera.
3. Bitácora pormenorizada de bugs y depuración.
4. Respuestas teóricas y análisis comparativo (OCaml).
5. Bibliografía.
6. Conclusión.

1 – Introducción y fundamentos del diseño funcional adoptado

En lugar de estructurar el software mediante un enfoque imperativo tradicional, el sistema se realizó mediante lo estipulado por la catedra:

Inmutabilidad Absoluta

El código desarrollado se realizó sin la utilización de variables globales mutables (`defparameter`, `defvar`) y de operadores de asignación destructiva (`setq`, `setf`). El estado **fluye como argumento inmutable** a través de las llamadas de funciones. El tiempo en el programa no se guarda en una variable mutable, sino que este se representa mediante un número entero estático representado en el argumento (`tiempoUnix`) el cual simula el paso del tiempo llamando a la función con números cada vez más grandes.

Transparencia Referencial y Funciones Puras

Las funciones del semáforo no dependen de variables globales ni de factores externos del hardware, y no modifican nada en la memoria. Permitiendo así que en el ingreso de datos la función me devuelva siempre el mismo resultado. Esto hace que el sistema de semáforos sea totalmente predecible y seguro.

Procesamiento Declarativo de Listas

Para procesar los datos y armar los reportes de tiempo del semáforo, no usamos ningún bucle tradicional como `loop` o `dolist`.

En su lugar, resolvimos todo de manera declarativa con dos herramientas:

1. Listas de Asociación (`alists`): Para guardar los datos de forma segura y sin riesgo de que se alteren en memoria.
2. Operación de reducción funcional: Para determinar la duración total del ciclo vial sin recurrir a contadores ni acumuladores mutables, se utiliza una operación de reducción matemática directa mediante el operador `+`. Esta combina todos los tiempos extraídos de la lista de asociación en un único valor numérico final.
3. Funciones de Orden Superior: El uso de combinadores como `mapcar` y `reduce` permite proyectar transformaciones de forma matemática sobre colecciones de celdas `CONS`, reduciendo la complejidad del código y alineándolo con el formalismo del cálculo `lambda`.

2 - Justificación de herramientas y arquitectura de frontera.

Justificación de la elección de cl-json

Elegimos cl-json porque consideramos que es la opción óptima para gestionar los tiempos del semáforo. De esta manera, si en el futuro se necesita modificar la duración de una luz o de una intermitencia, se puede hacer directamente cambiando los números en el archivo config.json, sin necesidad de tocar ni una sola línea del código fuente.

Además, la librería respeta el principio de Inmutabilidad no utilizando variables globales mutables ni modifica la memoria con asignaciones destructivas (como setq o setf). Simplemente recibe el flujo de datos del archivo y nos devuelve una lista fija en memoria lista para usarse.

Por último, aísla la Entrada/Salida, esto significa que la impureza de leer el el archivo config.json se queda atrapada únicamente en esa función de carga, permitiendo que el núcleo de nuestro semáforo (timer) permanezca puro, intacto y libre de alteraciones.

Justificación de la utilización de SBCL sobre CLISP

Debido a problemas de incompatibilidad, ya que nuestros sistemas operativos (Windows 10/11) presentan un bug con algunas versiones de CLISP donde cada vez que se ejecuta el comando (load ...) este se confunde con los nombres largos de Windows y se rompe.

Entonces decidimos consultar con la IA y una de las soluciones más fáciles que nos ofreció fue la utilización de SBCL el cual es mas actual y Windows lo detecta casi de forma automática y sin errores raros.

Aislamiento de la Entrada/Salida

Los procesos de Entrada/Salida son impuros por definición. Para evitar esta impureza en el sistema se realizó:

- Una vez recuperados los datos del archivo, el flujo de Entrada/Salida se cierra y la lista se vuelve inmutable en memoria. Esta estructura limpia es enviada al (timer), el cual procesa los datos de forma abstracta, predecible y protegida contra fallas externas.

3 – Bitácora pormenorizada de bugs y depuración

Al ejecutarse la función timer se producía un error el cual se debía a que timer ya es un símbolo reservado e interno de SBCL (específicamente se

encuentra en el paquete SB-EXT para manejar temporizadores del sistema). Como los paquetes internos de SBCL están “bloqueados” para evitar que se rompa el entorno sin querer, el compilador se frena.

La solución más simple a este problema fue simplemente cambiar el nombre de timer por timer-semaforo y no hubo más inconvenientes, provocando que se ejecutase sin problemas

4 - Respuestas teóricas y análisis comparativo (OCaml)

Al igual que Haskell, OCaml usa tipos estáticos, pero cuenta con *Inferencia de Tipos*. ¿Tuvieron que declarar explícitamente de qué tipo eran las funciones o el compilador lo dedujo solo? Muestren cómo lo interpreta el entorno.

A diferencia del entorno inyectado dinámicamente de Common Lisp donde los tipos de datos están ligados a los valores en ejecución y su consistencia se comprueba en tiempo de ejecución, OCaml opera bajo un esquema de tipado estático y fuerte. El cien por cien de las comprobaciones de consistencia de tipos se realizan en tiempo de compilación.

En la migración a OCaml no fue necesario declarar explícitamente el tipo de dato de ninguna variable o parámetro. Gracias al algoritmo de inferencia de tipos (Hindley-Milner), el compilador analiza matemáticamente el flujo de control, los operadores utilizados y las ramificaciones para deducir de forma autónoma las firmas exactas más generales posibles.

- Comprobación en transición: Al aplicar estructuras sintácticas, el sistema infiere inmediatamente los tipos correspondientes de los argumentos de entrada y determina que el retorno está conformado por un tipo variante y una cadena de texto (`color_actual * string`).
- Comprobación en timer: El operador `mod` en OCaml está restringido por diseño únicamente a operandos numéricos enteros (`int -> int -> int`). Al compilar la expresión `timestamp mod duracion_total`, el compilador deduce que el parámetro `timestamp` es estrictamente de tipo `int`, forzando la firma de la función. Ejemplo:

OCaml es un lenguaje "orientado a expresiones". ¿Cómo cambia esto la estructura de control de flujos (como el uso de `match...with`) comparado con los condicionales de Lisp?

Que OCaml sea "orientado a expresiones" significa que **toda construcción sintáctica computa, reduce y devuelve un valor de forma directa**. A

diferencia de los lenguajes imperativos basados en "sentencias" (instrucciones que ejecutan acciones abstractas sin retornar datos), en OCaml el flujo de control no decide qué comandos ejecutar, sino qué subexpresión matemática va a resolver y reducir el flujo a un valor final unificado.

Al analizar el modelado de un sistema de control de tráfico, las diferencias operativas entre las macros condicionales de Common Lisp y el *Pattern Matching* de OCaml revelan un salto en seguridad y diseño computacional:

Common Lisp (*cond* / *case*): Evalúan predicados de manera puramente secuencial. Aunque retornan el valor de la última expresión de la rama seleccionada, permiten efectos secundarios implícitos. Además, si ninguna condición se cumple, Lisp devuelve dinámicamente *NIL*, trasladando el peligro de un estado imprevisto al tiempo de ejecución.

OCaml (*match...with*): Es una estructura de coincidencia de patrones que actúa como una expresión unificada. Al estar integrada en un entorno fuertemente tipado en el compilador, introduce dos restricciones matemáticas estrictas:

Exhaustividad Obligatoria: El compilador analiza estáticamente que *todos* los casos posibles tengan una rama asignada. Si se omite un estado, el compilador arroja un error (*non-exhaustive match*) e impide la ejecución del programa, eliminando fallos lógicos latentes.

Consistencia de Tipos: Al ser una expresión, el principio de inferencia de tipos exige que **todas las ramas retornen exactamente el mismo tipo de dato**, garantizando una coherencia absoluta en la salida.

Mientras que en Common Lisp el flujo se controla evaluando secuencialmente si una condición lógica es verdadera para ejecutar un camino, OCaml transforma el control de flujo en un mapeo de patrones declarativos e inmutables. Esto inmuniza matemáticamente al sistema ante estados inválidos antes de que el programa llegue a ejecutarse.

Dificultades técnicas al desarrollar códigos

Durante el desarrollo en Ocaml, el grupo presentó inconvenientes de compatibilidad y configuración del entorno de Ocaml en Windows. Como alternativa, se trabajó con el intérprete proporcionado por el profesor, *Ocaml Playground*. Mediante esta herramienta fue posible desarrollar, probar y ejecutar correctamente los programas implementados, permitiendo verificar su funcionamiento y validar los resultados obtenidos.

5 – Bibliografía y referencias

<https://cl-json.common-lisp.dev/cl-json.html>

<https://www.sbcl.org/manual/sbcl.pdf>

<https://ocaml.org/docs/values-and-functions>

6 – Conclusión

En este trabajo desarrollamos un sistema de semáforo inteligente aplicando los principios de la programación funcional mediante Common Lisp y OCaml. Se implementó la lógica de transición de estados, el temporizador basado en ciclos temporales, el sistema de auditoría mediante registros de eventos y las herramientas de análisis para el estudio del comportamiento temporal del semáforo.

Durante el desarrollo se respetaron conceptos fundamentales del paradigma funcional, evitando el uso de variables globales mutables y privilegiando funciones puras, estructuras no destructivas y el flujo de datos a través de parámetros. Además, la incorporación de estados intermitentes permitió extender el modelo original para representar situaciones de seguridad vial de manera más realista.

La segunda fase permitió desacoplar la configuración de la lógica del programa mediante el uso de archivos JSON externos, facilitando la modificación de parámetros sin necesidad de alterar el código fuente. Finalmente, la implementación de componentes equivalentes en OCaml permitió comparar dos lenguajes funcionales con características diferentes, observando especialmente las ventajas del tipado estático y la inferencia de tipos de OCaml frente al tipado dinámico de Lisp.

Como resultado, se obtuvo una solución modular, extensible y mantenible, capaz de simular el comportamiento de un semáforo inteligente y de servir como caso práctico para el estudio de técnicas de programación funcional en distintos lenguajes.